UNIVERSITY OF PISA
MASTER'S DEGREE IN CYBERSECURITY

SIMPLIFIED RC4 STREAM CIPHER V2 (DECRYPTION)

COURSE HARDWARE AND EMBEDDED SECURITY

Author(s)
Giovanni Neglia

Academic year 2023/2024

Contents

CHAPTER 1

Project Specifications

The project requirment was to design a top-level module implementing the decryption capabilities of the RC4 algoritgm. This algorithm uses a symmetric key (the same for both encryption and decryption) to generate the keystream through two main phases: the Key Scheduling Algorithm (KSA) and the Pseudo-Random Generator Algorithm (PRGA). The original version of RC4 supports a key length ranging from 1 to 256 bytes, but in our simplified case study, the key length is fixed at 32 bytes (256 bits). In the following sections, we will analyze the two main components of the RC4 algorithm, focusing on the relevant details for our implementation:

```
1     for i from 0 to 255
2         S[i] := i
3     endfor
4     j := 0
5     for i from 0 to 255
6         j := (j + S[i] + key[i mod 32]) mod 256
7         swap values of S[i] and S[j]
8     endfor
```

Figure 1.1: *KSA pseudo-code*

The code listing in Figure 1.1 presents the pseudo-code of the KSA. This algorithm updates two counters (i and j) at each iteration before swapping the contents of the array at the positions indexed by them. In particular, the computation of the j counter is the step that incorporates the key. Once this process is completed, the first permutation phase of RC4 is finalized.

```
i := 0
j := 0
while GeneratingOutput:
    i := (i + 1) mod 256
    j := (j + S[i]) mod 256
    swap values of S[i] and S[j]
    K[1] := S[(S[i]+S[j]) mod 256]
    P[1] := C[1] XOR K[1]
    output K
endwhile
```

Figure 1.2: *PRGA pseudo-code*

The second phase, illustrated in Figure 1.2, consists of the PRGA algorithm, which applies a second permutation before decrypting the ciphertext. Again, the counters i and j are updated to determine the correct permutation. The module then computes the K value based on the two counters and the S array, applying an XOR operation to retrieve the original plaintext. The while loop in the pseudocode represents the stream nature of RC4, but in our implementation, it is replaced by a for loop iterating over the length of the ciphertext. Given this high-level overview of the required module, the design has been structured as follows:

- Given the ciphertext, the key, and the corresponding plaintext, the decryption module must produce a matching output.
- The design must include an asynchronous active-low reset port.
- The module must implement an input flag din_valid that is set to 1 when the input is stable and 0 otherwise.
- Similarly, an output flag *dout_ready* must be implemented, where 1 indicates a stable output and 0 otherwise.

To meet these specifications, the top-level module is defined as shown in Figure 1.3:

```
module rc4_dec_module (
  input        clk,
  input          rst_n,
  input [7:0]    key [31:0],
  input [7:0]    din, // Ciphertext
  input          din_valid,
  output  reg      dout_ready,
  output  reg [7:0] dout, // Plaintext
  output  reg      init_done
);
```

Figure 1.3: *Top-level module declaration*

CHAPTER 2

High-level Model

The high level has been implemented in Python, creating a script that performs encryption on a series of plaintexts using the RC4 algorithm. The script defines a global variable to set the key length to the desired size and then executes the KSA and PRGA phases to generate the corresponding ciphertexts. Both keys and plaintexts are hard-coded within the script and later saved as binary sequences in the *modelsim/tv* directory. These stored sequences serve as test vectors for the testbenches. The same process is applied to the computed ciphertexts, ensuring that all data is properly stored for verification. As illustrated in Figure 2.1, the RC4 encryption algorithm is divided into two separate functions, each representing one of the two main phases:

- The KSA phase, responsible for key scheduling.
- The PRGA phase, which generates the keystream and performs the encryption.

. Additionally, the PRGA function appends the encrypted stream output to an array that represents the ciphertext obtained from the given plaintext. The lines of code shown in Figure 2.2 execute the script and store the results in a set of four files, each containing data for one of the involved elements.

```python
MOD = 256
KEY_LENGHT = 32

# KEY SCHEDULING ALGORITHM
def ksa(key):

    # Creazione array
    S = list(range(MOD))
    j = 0
    for i in range(MOD):
        j = (j + S[i] + ord(key[i % KEY_LENGHT])) % MOD
        # Swap
        S[i], S[j] = S[j], S[i]

    return S

# PSEUDO-RANDOM GENERATOR ALGORITHM and ENCRYPTION
def prga(S, plaintext):

    # Plaintext
    P = [ord(p) for p in plaintext]
    # Ciphertext
    C = []

    i = 0
    j = 0
    for l in range(0, len(P)):
        i = (i+1) % MOD
        j = (j+ S[i]) % MOD
        # Swap
        S[i], S[j] = S[j], S[i]

        K = S[(S[i]+S[j]) % MOD]
        C.append(P[l] ^ K)

    return C, P
```

Figure 2.1: *Pyhton KSA and PRGA implementation*

```python
    i = 0
    for key,plaintext in zip(keys,plaintexts):
        # Registrazione chiave
        with open(path + f"key{i}.txt", 'w') as f:
            f.write('\n'.join(['{0:08b}'.format(ord(c)) for c in key]))

        # RC4
        S = ksa(key)
        C, P = prga(S,plaintext)

        # Registrazione ciphertext
        with open(path + f"cipher{i}.txt", 'w') as f:
            f.write('\n'.join(['{0:08b}'.format(c) for c in C]))

        # Registrazione plaintext
        with open(path + f"plain{i}.txt", 'w') as f:
            f.write('\n'.join(['{0:08b}'.format(p) for p in P]))

        i += 1
```

Figure 2.2: *Script logic*

CHAPTER 3

RTL Design

Figure 3.1 shows the block diagram of the top-level architecture, illustrating the input, output, and internal signals involved in the decryption process.

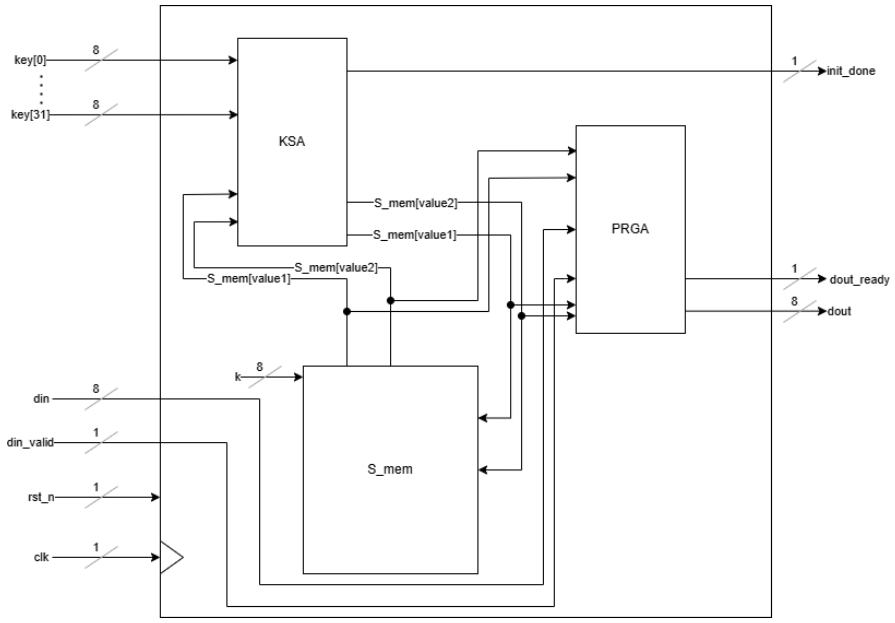


Figure 3.1: *Module block diagram scheme*

The design follows the standard RC4 algorithm, which is divided into two main phases:

- The KSA block initializes the S_mem (state memory) using the 32-byte key input. It performs a series of permutations on the state array and, once completed, asserts

the init_done signal, indicating that the system is ready to proceed to the decryption phase. The KSA interacts with the S_mem through read and write operations based on indices derived from an internal counter.

- Once the initialization is complete, the PRGA block takes as inputs the din (ciphertext byte) and din_valid (indicating valid input data). Using the continuously updated values of S_mem, it generates a keystream byte, which is XORed with the input ciphertext to produce the dout (plaintext output). The dout_ready signal is asserted when valid data is available.

- The S_mem block stores the 256-byte permutation table used by both the KSA and PRGA. The state transitions involve continuous swapping of elements at positions i and j, as dictated by the RC4 algorithm. The PRGA updates i and j dynamically, influencing the keystream generation and decryption process.

This structured approach ensures the correct execution of the decryption process while optimizing resource usage.

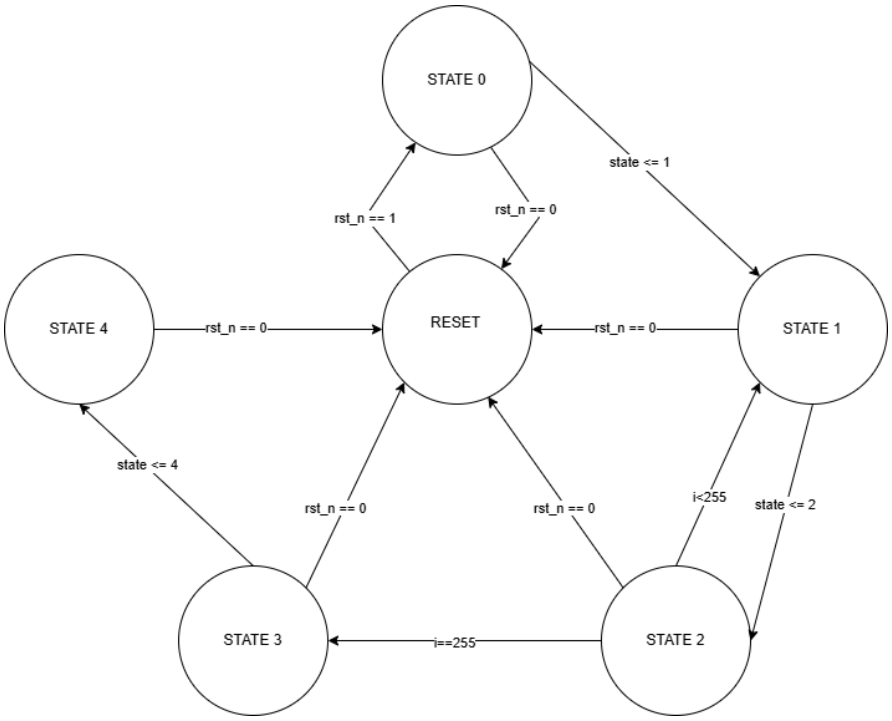


Figure 3.2: *Finite State Machine representation*

The finite state machine presented in Figure 3.2 illustrates the developed RC4 implementation:

- **RESET**, the startup state. In this state, *dout* is set to zero, *dout_ready* is deasserted (set to 0), *init_done* is also set to 0.

- **STATE 0**, initializes the *S_mem* array along with the two counters *i* and *j*.

- **STATE 1**, computes the value of *j* using the provided key.

- **STATE 2**, executes the swap operation of the KSA algorithm. Additionally, the value of *i* is checked to determine the next state:
  - If the permutation loop is complete, the state transitions to STATE 3, setting $i = 1$ (its first PRGA value) and initializing *j* accordingly.
  - Otherwise, the state returns to STATE 1 to continue the permutation process.

  .
- **STATE 3**, performs the swap operation of the PRGA algorithm. In this state, the *S_ij* value is computed, which will be used in the next state to index *S_mem*. Additionaly, *i* is incremented, and the *init_done* signal is asserted.
- **STATE 4**, executes the final decryption process. If *din_valid* is asserted, the *din* value is XORed with the corresponding value in *S_mem* indexed by *S_ij*. Meanwhile, the necessary variables for the next cycle are updated:
  - *i* is incremented.
  - *j* is recomputed.
  - The final swap operation is performed, considering the newly updated values.
  - lastly, the new index *S_ij* is computed for the next XOR operation.

CHAPTER 4

Interface Specifications and Expected Behavior

The RC4 decryption module has the following input and output ports, shown in Figure 4.1. The module operates as a byte-oriented streaming system, processing one byte of ciphertext at a time and producing the corresponding plaintext output.

Signal Name	Direction	Width	Description
clk	Input	1 bit	System clock
rst_n	Input	1 bit	Active-low asynchronous reset
key[31:0]	Input	32x8 bits	32-byte key for decryption
din	Input	8 bits	Ciphertext byte input
din_valid	Input	1 bit	Flag indicating valid input data
dout	Output	8 bits	Decrypted plaintext byte
dout_ready	Output	1 bit	Flag indicating valid output data
init_done	Output	1 bit	Indicates completion of key scheduling

Table 4.1: *RC4 Decryption Module Interface*

Upon startup, the module requires an initialization phase where the Key Scheduling Algorithm (KSA) is executed. This phase follows these steps:

- Reset activation (rst_n = 0) clears all internal states.
- When rst_n is deasserted (rst_n = 1), the module starts computing the S array permutation using the provided key.
- The process completes when init_done is set to 1, indicating that decryption can begin.

At this stage, dout_ready remains 0, as no decryption is performed during initialization. Once the initialization is complete (init_done = 1), the module enters the decryption phase, which operates as follows:

- The user provides one ciphertext byte at a time through din, ensuring that din_valid = 1 to signal valid input.
- The module processes the byte using the Pseudo-Random Generation Algorithm (PRGA).
- After one clock cycle, dout_ready is set to 1, and dout contains the corresponding plaintext byte.
- The output remains valid as long as dout_ready = 1, allowing the user to read the decrypted data.

The decryption continues for each new din byte, ensuring that din_valid is asserted whenever new data is provided. To illustrate the expected behavior of the module, the Figure 4.1 and Figure 4.2 show the waveforms to describe the key phases of operation.

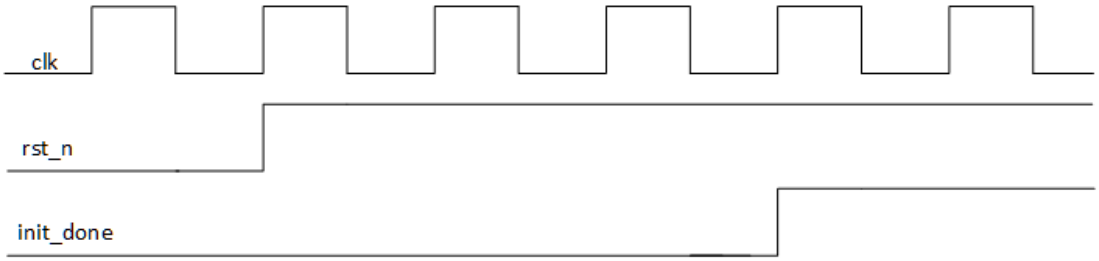


Figure 4.1: *Waveform 1: Reset and Initialization*

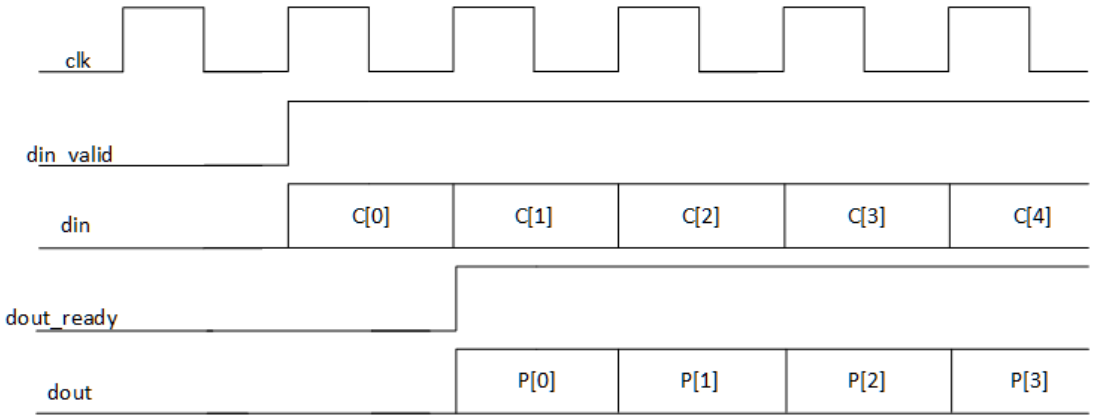


Figure 4.2: *Waveform 2: Normal Decryption Operation*

Waveform 1 highlights reset and initialization phase:

- rst_n = 0 → Module resets.
- rst_n = 1 → KSA starts processing.
- After key scheduling, init_done is set to 1, indicating readiness.

Waveform 2 highlights a decryption operation:

- din_valid = 1 indicates new ciphertext input.

- After one cycle, dout outputs the corresponding plaintext byte.
- dout_ready = 1 signals that output is stable and ready to be read.

In Figure 4.3 the expected output following a correct decryption executed by the module is provided. The output is formatted as follow:

```
VSIM 2> run -all
#  0| Cipher: 110111    Decoded: 1001100   Expected: 1001100   dout_ready: 1 OK
#  1| Cipher: 111100    Decoded: 1101111   Expected: 1101111   dout_ready: 1 OK
#  2| Cipher: 101001    Decoded: 1110010   Expected: 1110010   dout_ready: 1 OK
#  3| Cipher: 11010011  Decoded: 1100101   Expected: 1100101   dout_ready: 1 OK
#  4| Cipher: 10011010  Decoded: 1101101   Expected: 1101101   dout_ready: 1 OK
#  5| Cipher: 10111101  Decoded: 100000    Expected: 100000    dout_ready: 1 OK
#  6| Cipher: 101111    Decoded: 1101001   Expected: 1101001   dout_ready: 1 OK
#  7| Cipher: 10101100  Decoded: 1110000   Expected: 1110000   dout_ready: 1 OK
#  8| Cipher: 10001100  Decoded: 1110011   Expected: 1110011   dout_ready: 1 OK
#  9| Cipher: 1011111   Decoded: 1110101   Expected: 1110101   dout_ready: 1 OK
# 10| Cipher: 1000111   Decoded: 1101101   Expected: 1101101   dout_ready: 1 OK
# 11| Cipher: 11111011  Decoded: 100000    Expected: 100000    dout_ready: 1 OK
# 12| Cipher: 11010101  Decoded: 1100100   Expected: 1100100   dout_ready: 1 OK
# 13| Cipher: 10000010  Decoded: 1101111   Expected: 1101111   dout_ready: 1 OK
# 14| Cipher: 1110001   Decoded: 1101100   Expected: 1101100   dout_ready: 1 OK
# 15| Cipher: 1111111   Decoded: 1101111   Expected: 1101111   dout_ready: 1 OK
# 16| Cipher: 10001111  Decoded: 1110010   Expected: 1110010   dout_ready: 1 OK
# 17| Cipher: 10011001  Decoded: 100000    Expected: 100000    dout_ready: 1 OK
# 18| Cipher: 110011    Decoded: 1110011   Expected: 1110011   dout_ready: 1 OK
# 19| Cipher: 11001010  Decoded: 1101001   Expected: 1101001   dout_ready: 1 OK
# 20| Cipher: 1011011   Decoded: 1110100   Expected: 1110100   dout_ready: 1 OK
# 21| Cipher: 10000000  Decoded: 100000    Expected: 100000    dout_ready: 1 OK
# 22| Cipher: 10010001  Decoded: 1100001   Expected: 1100001   dout_ready: 1 OK
# 23| Cipher: 1010110   Decoded: 1101101   Expected: 1101101   dout_ready: 1 OK
# 24| Cipher: 11000010  Decoded: 1100101   Expected: 1100101   dout_ready: 1 OK
# 25| Cipher: 10110011  Decoded: 1110100   Expected: 1110100   dout_ready: 1 OK
# Test completato con successo!
# ** Note: $stop    : C:/Users/negli/Desktop/Giovanni/Magistrale/Secondo Semestre/HES/Progetto/tb/simplified_rc4_dec_module_tb0.sv(93)
#    Time: 5415 ps  Iteration: 1  Instance: /rc4_dec_module_tb
# Break in Module rc4_dec_module_tb at C:/Users/negli/Desktop/Giovanni/Magistrale/Secondo Semestre/HES/Progetto/tb/simplified_rc4_dec_module_tb0.sv line 93

VSIM 3>
```

Figure 4.3: *Testbench 0 transcript output*

- Byte index value starting from 0
- Ciphertext byte binary value.
- Plaintext byte decrypted binary value
- Expected plaintext byte binary value
- *dout_ready* value
- OK or ERROR print according to module correct/incorrect behaviour

CHAPTER 5

Functional Verification

The 4 testbenches present in the corresponding folder *tb* follow the same structural logic, differing only in the values used for importing test vectors (ciphertext, key, and plaintext) and in specific loop parameters tailored to those values. Each testbench is designed to verify the correct decryption of an input ciphertext using a predefined key and compare the output with the expected plaintext. Since the module under test is an RC4-based symmetric-key decryption module, the verification aims to ensure:

- Correctness of the key scheduling algorithm (KSA): Ensuring that the key initialization process correctly sets up the internal state (S-array) used for decryption.
- Correctness of the pseudo-random generation algorithm (PRGA): Verifying that the key stream generated for decryption matches the expected sequence.
- Accuracy of decryption: Checking whether the decrypted output (dout) matches the expected plaintext for a given ciphertext and key.
- Proper synchronization and control signals: Ensuring that the dout_ready signal is asserted correctly and that data processing occurs in a stable manner.

The code present in the Figure 5.1 shows to us the *fork-join* structure to handle input feeding and output verification in parallel:

- Input Handling Process:
  - Reads test vectors (key, ciphertext, plaintext) from external files.
  - Waits for the module initialization (init_done signal).
  - Feeds the ciphertext byte-by-byte into the decryption module (din) while tracking the index of each input byte.

- Output Verification Process:
  - Waits for valid decrypted outputs (dout_ready signal).
  - Extracts the corresponding ciphertext index from a dynamic array.
  - Compares the decrypted output (dout) with the expected plaintext.
  - If a mismatch is detected, the simulation halts with an error message; otherwise, the process continues until all bytes are verified.

```
1   fork
2     begin
3       for(i = 0; i < 26; i++) begin
4         din_valid = 1;
5         din = ciphertext0[index1];
6
7         @(posedge clk);
8         wait (dout_ready == 1);
9
10        byte_index.push_back(index1);
11        index1 += 1;
12      end
13      din_valid = 0; // Finito la lettura
14    end
15
16    begin
17      @(posedge clk);
18      wait (dout_ready == 1);
19      for(j = 0; j < 26; j++) begin
20        @(posedge clk);
21
22        if (byte_index.size() > 0) begin
23          index2 = byte_index.pop_front();
24        end
25        else begin
26          $display("Errore: byte_index vuoto!");
27          $stop;
28        end
29
30        // Display per il confronto
31        $display("%2d| Cipher: %-8b  Decoded: %-8b  Expected: %-8b  dout_ready: %1d %-5s",
32          index2, ciphertext0[index2], dout, plaintext0[index2],
33          dout_ready, (plaintext0[index2] === dout && dout_ready) ? "OK" : "ERROR");
34
35        // Verifica la correttezza dell'output
36        if(plaintext0[index2] !== dout || 1 != dout_ready) begin
37          $display("Errore al byte %2d: Decodifica errata!", index2);
38          $stop;
39        end
40      end
41      $display("Test completato con successo!");
42      $stop;
43    end
44  join
```

Figure 5.1: *Testbench fork-join block*

The testing methodology follows a test vector approach, where known inputs (ciphertext and key) are compared against expected outputs (plaintext). The correctness of the decryption process is evaluated based on direct byte-by-byte comparisons.

CHAPTER 6

FPGA Implementation Results

Now, we proceed to present the results obtained using Quartus. The software version used was 20.1.1, and the selected device was 5CGXFC9D6F27C7, which belongs to the Cyclone V FPGA family. Figure 6.1 shows the Analysis/Synthesis Summary, where the resource utilization for implementing the module is approximately 5%. This suggests that a less powerful FPGA could be used to avoid resource waste, or alternatively, the decryption module could be integrated as a sub-module in a larger project. Figure 6.2 presents the Fitter Summary, highlighting the following aspects::

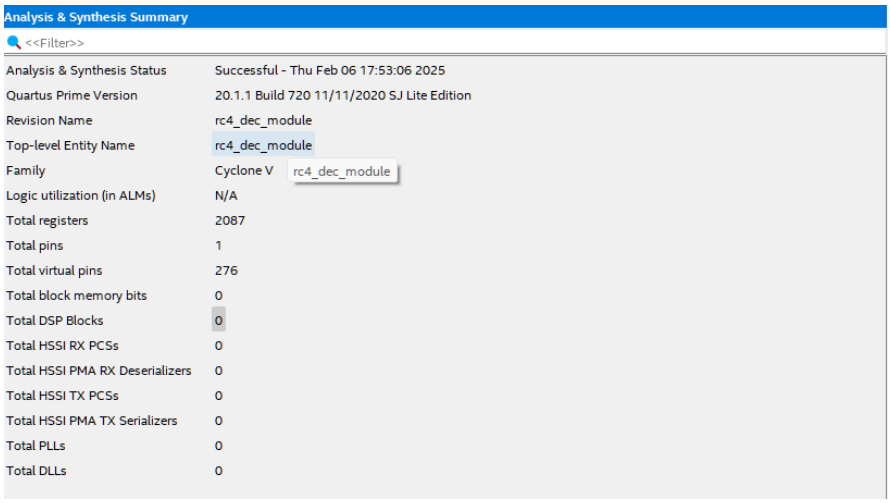
- Under the *Resource usage summary*, the total amount of logic resources utilized is displayed, providing a rough cost estimation of the module design.
- The Adaptive Logic Modules (ALMs) usage is further broken down into: [A] ALMs used for both LUTs and registers (1040 in this case); [B] ALMs used only for LUTs (4715); [C] ALMs used only for registers (4).

The next step involved performing Static Timing Analysis (STA) to determine the maximum operating frequency. This was done using a provided .sdc file template to set timing constraints.. As shown in Figure 6.3, through a trial-and-error approach, we determined a possible maximum frequency of 63.7 MHz (1/15.7 ns clock period). With this value, no timing violations were detected in the Quartus timing analysis report. Figure 6.4 and Figure 6.5 present the STA results under two different conditions:

- At 85°C, the estimated fmax is 64.89 MHz.
- At 0°C, the estimated fmax is 65.87 MHz.

As discussed during the course, the worst-case value must be considered to ensure the module functions correctly under all conditions. Quartus evaluates four different operating condition scenarios (Process, Voltage, and Temperature (PVT) corners) which

affect the propagation delays of logic cells. This explains why the report excludes "fast" scenario values, and why the maximum frequency is ultimately constrained by the high-temperature case.

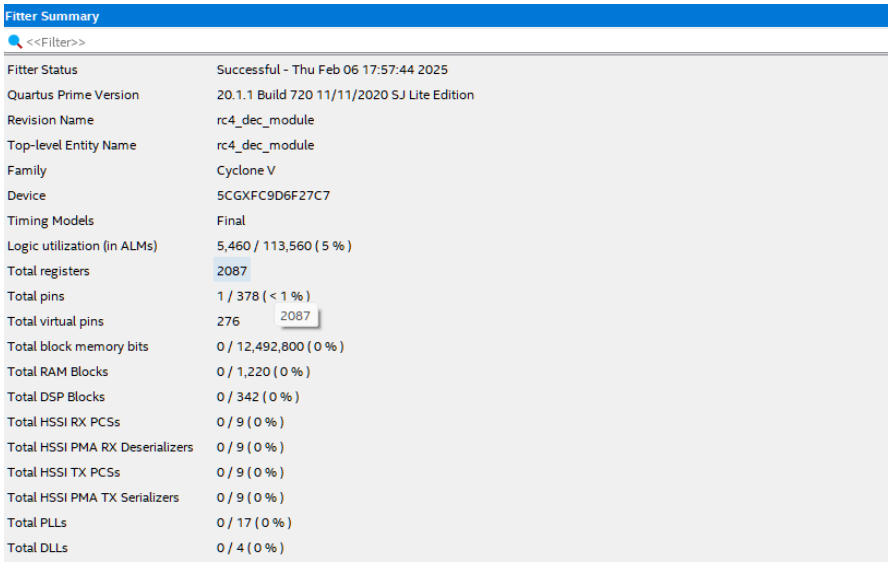


Analysis & Synthesis Summary

<<Filter>>

Analysis & Synthesis Status	Successful - Thu Feb 06 17:53:06 2025
Quartus Prime Version	20.1.1 Build 720 11/11/2020 SJ Lite Edition
Revision Name	rc4_dec_module
Top-level Entity Name	rc4_dec_module
Family	Cyclone V
Logic utilization (in ALMs)	N/A
Total registers	2087
Total pins	1
Total virtual pins	276
Total block memory bits	0
Total DSP Blocks	0
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0
Total DLLs	0

Figure 6.1: *Analysis and Synthesis summary from Quartus*



Fitter Summary

<<Filter>>

Fitter Status	Successful - Thu Feb 06 17:57:44 2025
Quartus Prime Version	20.1.1 Build 720 11/11/2020 SJ Lite Edition
Revision Name	rc4_dec_module
Top-level Entity Name	rc4_dec_module
Family	Cyclone V
Device	5CGXFC9D6F27C7
Timing Models	Final
Logic utilization (in ALMs)	5,460 / 113,560 (5 %)
Total registers	2087
Total pins	1 / 378 (< 1 %)
Total virtual pins	276
Total block memory bits	0 / 12,492,800 (0 %)
Total RAM Blocks	0 / 1,220 (0 %)
Total DSP Blocks	0 / 342 (0 %)
Total HSSI RX PCSs	0 / 9 (0 %)
Total HSSI PMA RX Deserializers	0 / 9 (0 %)
Total HSSI TX PCSs	0 / 9 (0 %)
Total HSSI PMA TX Serializers	0 / 9 (0 %)
Total PLLs	0 / 17 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 6.2: *Fitter summary from Quartus*

```
set CLK_PERIOD_NS 15.7
set MIN_IO_DELAY [expr double($CLK_PERIOD_NS)/10.0]
set MAX_IO_DELAY [expr double($CLK_PERIOD_NS)/5.0]
create_clock -name clk -period $CLK_PERIOD_NS [get_ports clk]
set_false_path -from [get_ports rst_n] -to [get_clocks clk]
set_input_delay -min $MIN_IO_DELAY [all_inputs] -clock [get_clocks clk]
set_input_delay -max $MAX_IO_DELAY [all_inputs] -clock [get_clocks clk]
set_output_delay -min $MIN_IO_DELAY [all_outputs] -clock [get_clocks clk]
set_output_delay -max $MAX_IO_DELAY [all_outputs] -clock [get_clocks clk]
```

Figure 6.3: *.sdc file*

Slow 1100mV 85C Model Fmax Summary

<<Filter>>

	Fmax	Restricted Fmax	Clock Name	Note
1	64.89 MHz	64.89 MHz	clk	

Figure 6.4: *Slow 1.1V 85°C model max frequency*

Slow 1100mV 0C Model Fmax Summary

<<Filter>>

	Fmax	Restricted Fmax	Clock Name	Note
1	65.87 MHz	65.87 MHz	clk	

Figure 6.5: *Slow 1.1V 0°C model max frequency*